Лекция 03 — Преобразования типов и CRTP

Содержание

1. Преобразования типов
    1. Стандартные преобразования
    2. Неявные преобразования
    3. Явные преобразования
        1. Явные преобразования в Си
        2. `const_cast`
        3. `static_cast`
        4. `dynamic_cast`
        5. `reinterpret_cast`
        6. Си-подобный и функциональный касты
        7. Снова о хороших манерах
2. CRTP

”Один раз у меня горел офис. В офисе сидели вот такие же чудаки, как вы, пока там горела тысяча квадратных метров. Говорили: «Щас я письмо допишу, закоммичу и пойду»”

— А. П. Хвастунов

Преобразования типов

С идеей приведения типов в статически типизированных языках, коим являются Си и C++, читатель знаком уже достаточно хорошо, да и знакомиться там, в общем-то, не с чем: тип переменной измениться не может, а время от времени нам всё же нужно воспринимать одну и ту же информацию в рамках ограничений разных типов — для этого нам, собственно, преобразования типов и нужны. Однако когда дело доходит до практической реализации этой идеи, у нас внезапно появляется множество вопросов, стоящих отдельного обсуждения, которое мы начнем с некоторой формализации понятия преобразования.

Стандартные преобразования

В качестве небольшой исторической справки обратимся к стандарту C17. Язык Си реализует приведение типов при помощи *стандартных преобразований*, среди которых можно выделить:

I. **Повышающие преобразования** (*integer/floating-point promotions*)

Повышающие преобразования происходят в соответствии со следующей иерархией, никак не изменяя значение переменной:

- `bool / char / short int -> int -> long int -> long long int`
- `float -> double`

Помимо прочего, имеет место приведение `enum` к любому целочисленному типу не младше ассоциированного с ним, и наоборот: любое целочисленное значение не младше, чем ассоциированное с перечислятелем, можно привести к нему.

II. **Понижающие преобразования** (*integer/floating-point conversions*)

При любом преобразовании между целыми или вещественными типами, не являющимся повышающим, новое значение вычисляется по модулю вместимости результирующего типа для беззнаковых и в implementation-defined манере для знаковых типов.

III. **Вещественно-целочисленные преобразования** (*floating-integral conversions*)

При приведении вещественного значения к целому типу его дробная часть отсекается. При приведении целого значения к вещественному типу его значение не изменяется, если это возможно, или, в противном случае, приводится к ближайшему доступному в implementation-defined манере.

Если при любом из этих преобразований преобразуемое значения не соответствует целевому диапазону, поведение не определено.

IV. **Преобразования указателей** (*pointer conversions*)

Нулевой указатель может быть приведён к любому типу, в результате чего получается константа `nullptr` этого типа.
Любой указатель может быть приведён к указателю на тип `void` и наоборот.

V. **Булевы преобразования** (*boolean conversions*)

Если значение переменной скалярного типа равно `0`, при приведении к `bool` результирующим значением выступит `false`; во всех остальных случаях им выступит `true`.

VI. **Типичные арифметические преобразования** (*usual arithmetic conversions*)

За этим названием скрывается паттерн, реализующий логику приведения типов операндов в скалярных бинарных выражениях и почти всегда заключающийся в

> повышающем преобразовании «младшего» типа из двух.

Эти и некоторые другие[1] стандартные преобразования объединяются в цепочки, что позволяет приводить даже те типы, которые не связаны непосредственно (например, `char` в `long`).

В ситуациях, когда некоторое выражение встречается в контексте, который не принимает его тип, компилятор может самостоятельно использовать описанные выше механизмы, к которым в C++ также добавляются цепочки пользовательских преобразований.
В частности, неявные преобразования выполняются:

- при передаче аргументов в функцию или оператор;
- при инициализации новых объектов;
- в условных конструкциях и циклах.

Явные преобразования

Явные преобразования в Си

Для того, чтобы пользователь мог самостоятельно использовать механизмы приведения типов, в языке Си определён уже отдалённо знакомый нам оператор каста, который может работать с любыми приводимыми скалярными типами:

```
int main() {
     float f = 5.2;
     int i = (int)f; // здесь произошёл явный floating -> integral каст
     printf("%i", i; // 5
}
```

Он, в общем-то, вполне неплохо справляется с поставленной задачей, особенно если вернуться лет так на 30 назад. Но — увы! — у него есть одна маленькая, но очень неприятная семантическая особенность.

Дело здесь в том, что унаследованный от Си оператор каста очень упёртый и крайне ответственно выполняет свою работу. Иными словами, с его помощью мы совершенно безнаказанно можем привести *что угодно к чему угодно* (в терминах скалярных типов, разумеется) — в частности, по той причине, что обратное преобразование из `void*` легально с точки зрения языка Си. Можно сделать, например, вот так:

```
int main() {
     char* s = "hvost238";
```

```
    printf("%i", *(int*)(void*)s); // какая-то глупость
}
```

...или вот так:

```
int main() {
    int i = 3104;
    printf("%s", (char*)(void*)i); // segfault
}
```

...или даже вот так:

```
int main() {
    const int i = 238;
    printf("%i", i);
    int* ptr = (int*)(&i);
    *ptr = 3104;
    printf("%i", i); // ???
}
```

И это только в Си — стоит ли говорить о том, чего можно было бы добиться с такой механикой в C++, когда у нас есть классы, исключения, вывод типов и проч. и проч.?

Это наблюдение порождает две мысли. Первая — давайте так не делать. Вторая — давайте делать как-нибудь не так, но по-другому, чтобы при этом не сильно от этого страдать. Из таких соображений в языке C++ появилось аж четыре новых выражения, позволяющих преобразовать типы в рамках заданных ограничений.

Явные преобразования

`const_cast`

`const_cast` — достаточно специфичное преобразование с единственной функцией: привести указатели или ссылки на одинаковые типы, сняв `cv`-квалификаторы:

```
int main() {
    int my_var = 5;
    const int& my_var_cref = my_var;
    int& my_var_ref = const_cast<int&>(my_var_cref);
}
```

Практически говоря, `const_cast` — вещь весьма бесполезная и редко используемая, но допустмиая для некоторых служебных финтов ушами. Например, если мы забыли про

существование квалификатора `mutable`, изменить состояние объекта в константном методе можно следующим образом:

```
class MyClass {
  public:
     void MyMethod() const {
          const_cast<MyClass*>(this)->conter_++;
     }

  private:
     int counter_ = 0;
};
```

С помощью `const_cast` можно провернуть тот же трюк с изменением константного объектиа, что был продемонстрирован выше в разговоре о кастах в Си. Его, впрочем, проворачивать не надо: это UB.

`static_cast`

Это преобразование рассматривает все допустимые цепочки неявных и пользовательских конверсий (за исключением снимающих квалификаторы — это прерогатива `const_cast`) В частности, выражение `static_cast<target-type>(expression)` корректно, если:

> 1. Мы приводим указатели из одной иерархии:
>
> ```
> Derived d;
> Base& d_ref = d;
> static_cast<Derived&>(ref);
> ```

> 2. Выражение `target-type tmp(expression)` корректно:
>
> ```
> struct A {};
> struct B {
     B(A a);
> };

> int main() {
     A a;
     static_cast<B>(a);
> }
> ```

> 3. Существует последовательность неявных преобразований из `expression` в `target-type`

```
enum MyEnum {
    odyn = 0,
    dva = 1,
    tri = 4
};

int main() {
    static_cast<MyEnum>(1); // dva
}
```

4. Существует последовательность неявных преобразований из `target-type` в `expression`

Этот принцип, в частности, означает, что `static_cast` позволяет приведение из `void*`:

```
int main(){
    int x = 5;
    int* xptr = &x;
    void* bad_ptr = static_cast<void*>(xptr);
    int* cast_back = static_cast<int*>(bad_ptr);
}
```

5. `expression` — такой же или менее квалифицированный тип, что и `target-type`

```
static_cast<const int>(int(52));
```

Практически говоря, `static_cast` — это достаточно умная версия оператора преобразования из Си, которая рассматривает только «безопасные» цепочки преобразований; тем не менее, у него всё равно есть одно уязвимое место.

При приведении базового класса к наследнику (разумеется, по указателю или по ссылке) мы в момент компиляции не имеем возможности убедиться, что это преобразование вообще валидно, то есть ответственность за эту гарантию лежит на пользователе. Если она не выполняется, преобразование всё равно выполнится, однако дальнейшее поведение не будет определено:

```
struct Base {};

struct Derived : Base {
    void Hello() {
        std::cout << x;
    }
```

```
    int x = 3104;
};

int main(){
    Base b;
    Derived d;
    Base& bref = b;
    Derived& dref = static_cast<Derived&>(bref);
    dref.Hello(); // UB!
}
```

`dynamic_cast`

Выражения типа `dynamic_cast` решают ровно описанную проблему, вертикально и горизонтально преобразуя указатели и ссылки по иерархии, но делая это не в момент компиляции, а во время выполнения программы, что позволяет использовать RTTI для определения валидности каста (откуда сразу следует важное замечание о том, что `dynamic_cast` требует наличия vtable при работе с иерархией наследования):

```
struct Base {
     virtual ~Base() = default;
};

struct Derived : Base {
    void Hello() {
        std::cout << x;
    }
    int x = 3104;
};

int main(){
    Base b;
    Derived d;
    Base& bref = b;
    Derived& dref = dynamic_cast<Derived&>(bref);
    dref.Hello(); // exception
}
```

При ошибке преобразования указателей `dynamic_cast` возвращает `nullptr`; при ошибке преобразования ссылок он выбрасывает `std::bad_cast`.

Помимо того, что `dynamic_cast` реализует *upcast* и *downcast* указателей и ссылок на полиморфные объекты, с помощью него также можно осуществить т.н. *crosscast*, то есть перекрёстно привести одну базу к другой:

```
struct A {
    virtual ~A() = default;
};
struct B {
    virtual ~B() = default;
};
struct C : A, B {
    C() = default;
};

int main() {
     C c;
     B& b_ref = c;
    dynamic_cast<A&>(b_ref);
}
```

Другое любопытное свойство `dynamic_cast` связно с указателем на `void` [2] : если целевой тип выражается им, то результатом будет указатель на иерархически самый младший объект, на который указывает приводимое выражение:

```
struct A {
    virtual ~A() = default;
};
struct B {
    virtual ~B() = default;
};
struct C : A, B {
    C() = default;
};

int main() {
     C c;
     B* ptr = &c;
    std::cout << std::boolalpha
              << (static_cast<void*>(ptr) == static_cast<void*>(&c)) <<
'\n';
    std::cout << std::boolalpha
              << (dynamic_cast<void*>(ptr) == static_cast<void*>(&c)) <<
'\n';
}
```

Кроме того, аналогично `static_cast`, он может добавлять квалифакторы (и не может их снимать).

`reinterpret_cast`

Но постойте! С этим умными кастами мы совершенно потеряли возможность совершать глупости.

`reinterpret_cast` позволяет приводить любые типы к любым — сравнительно так же, как касты в Си, — используя практически сырое побитовое представление данных. Указатель может быть приведён к любому достаточно вместительному целочисленному типу, и наоборот. Любые указатели (в т.ч. указатели на функции) и ссылки могут быть приведены друг к другу. В общем, совершенный ужас.

Что удивительно, так это то, что `reinterpret_cast` может даже быть полезен. Одно из его применений — это сохранение данных некоторой структуры в текстовый файл и последующее их чтение. Побайтовое представление позволяет нам не использовать .csv или, боже упаси, не выдумывать DSL для этой задачи, а просто записать всё как есть и считать тоже как есть. Такая программа будет категорически непереносима, однако если переносимость нам и не требуется, то это решение, наверное, одно из самых простых и удобных.

Си-подобный и функциональный касты

Помимо описанных выше выражений, в языке C++ существует ещё два синтаксически сходных между собой способа произвести преобразование типов:

Си-подобный каст (*он же C-style*)	`(type-id) expression`
Функциональный каст (*он же function-style*)	`type-id(expression)`

Си-подобный каст хотя и сохраняет семантику преобразующего оператора из Си, но не совсем его копирует — на то он и назван «подобным». Строго говоря, он последовательно применить одно из следующих преобразований до тех пор, пока это не станет возможным:

1. `const_cast`
2. `static_cast`
3. `static_cast` + `const_cast`
4. `reinterpret_cast`
5. `reinterpret_cast` + `const_cast`

Отсюда видно, этот каст сохраняет семантику своего предка и приводит типы всё с тем же упорством, однако использует для этого лишь наиболее подходящий механизм, как если бы мы сами делали это преобразовние.

Функциональный каст, как можно заметить, очень похож на *прямую инициализацию*, и, практически говоря, в большинстве случаев ей и является; впрочем, если в скобках содержится ровно одно выражение, то он эквивалентен соответствующему си-подобному касту.

Снова о хороших манерах

При всех ограничениях преобразующих выражений в C++ приведение типов всё равно остаётся не самым безопасным мероприятием. Совсем строго можно сказать, что в хорошо спроектированной программе кастов вообще быть не должно, но, реалистично говоря, добиться этого сложно.

Что однозначно — так это то, что не следует использовать Си-подобный и функциональный касты, ибо они хотя и выполняют свою задачу, но целиком скрывают за собой саму логику преобразования, о которой легко забыть и которая, помимо прочего, может быть совсем неизвестна стороннему читателю. Жестокая правда в том, что к нему всё ещё повсеместно прибегают, потому что написать `(int)x` ощутимо проще, чем `const_cast<int>(static_cast<const int>(x))`. Тем не менее, всё ещё настоятельно рекомендуется не лениться и писать чуть побольше букв, чтобы всем чуть получше жилось.

CRTP

Curiously Recurring Template Pattern (*любопытно зацикливающийся шаблонный паттерн*) — связанная с преобразованиями типов достаточно косвенно, но нашедшая своё место именно здесь идиома программирования на C++. Суть CRTP проста: базовый класс принимает своего наследника в качестве шаблонного аргумента.

```
template<typename T>
class Base {};

class Derived : public Base<Derived> {};
```

Поначалу это может показаться совершенной дуростью (*как, в частности, показалось в свое время автору*), однако вся прелесть CRTP раскрывается в деле. Сначала

рассмотрим пример попроще — реализацию базового класса, который будет считать количество созданных экземпляров наследника:

```
class Counter {
  public:
     Counter() {
          created_++;
     }
  private:
     static size_t created_ = 0;
};
```

Если теперь унаследоваться от `Counter`, то всё действительно будет работать... ровно до момента, пока мы не унаследуем от него ещё один класс и не обнаружим, что созданный счётчик является общим для всех производных классов, что, конечно, не соответствует нужной нам семантике. Если подкрутить сюда CRTP, мы получим следующую картину:

```
template<typename T>
class Counter {
  public:
     Counter() {
          created_++;
     }
  private:
     static size_t created_ = 0;
};

class Derived : public Counter<Derived> {};
```

Теперь для каждого класса создастся свой уникальный счётчик, поскольку общего `Counter` больше физически не существует.

Обратимся к другому, куда более интересному примеру использования CRTP:

```
template<typename Derived>
class ArithmeticsBase {
     Derived operator+(const Derived& other) {
          // some logic...
          return static_cast<Derived&>(*this);
     }
};
```

Если мы теперь унаследуемся от `ArithmeticsBase`, то сможем использовать логику определенного в нём оператора сложения так же, как если бы самостоятельно задали её для производного класса. Напоминает что-нибудь?
Мы получили самый настоящий *полиморфизм подтипов* и, что самое любопытное, получили его во время компиляции (*статический полиморфизм*), то есть чуть ли не задаром, особенно сравнительно оверхеда на виртуальные таблицы. Мы можем пойти в этом ещё дальше, позволив наследникам полиморфно работать со внешними функциями:

```
template<typename Derived>
class Animal {
  public:
     std::string Kto() const {
          return static_cast<const Derived*>(this)->Kto();
     }
};

class Bird : public Animal<Bird> {
  public:
     std::string Kto() const {
          return "pticka";
     }
};

class Dog : public Animal<Dog> {
  public:
     std::string Kto() const {
          return "@";
     }
};

template<typename T>
void YaKto(const Animal<T>& animal) {
     std::cout << "Ya " << animal.Kto() << '\n';
}

int main() {
     Dog dog;
     Bird bird;
     YaKto(dog); // Ya @
     YaKto(bird); // Ya pticka
}
```

...или реализовать что-то похожее на идиому NVI, всё также в compile-time:

```
template<typename Derived>
class Base {
    void DoSomething() {
        // pre-work
        static_cast<Derived*>(this)->DoSomethingImpl();
        // post-work
    }
};

template<typename T>
void DoSomethingFunc(Base<T> obj) {
    obj.DoSomething();
}
```

CRTP — специфичный, но очень мощный инструмент. Он впрочем, совершенно не заменяет собой виртуальные функции: как минимум потому, что код с CRTP достаточно неочевиден для понимания, а как максимум — потому что не предоставляет возможности создавать коллекции полиморфных объектов, что является одной из ключевых функций при работе с полиморфизмом подтипов.

Что касается производительности, то, например, на компиляторе GCC CRTP не оставляет виртуальным функциям никакого шанса; на clang они показывают схожие результаты, хотя в общем случае CRTP никогда не будет медленнее и, вероятнее всего, даже всегда будет быстрее. Его можно использовать как средство оптимизации, но подходить к этому, как и ко всему остальному в языке C++, нужно с умом и осторожностью.

Источники

1. А. П. Хвастунов — Лекции по основам программирования на C++, 2 семестр, 2026
2. ISO/IEC C17 §6.3 — Conversions (https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2347.pdf)
3. ISO/IEC C++20 §7.3 — Standard conversions (https://isocpp.org/files/papers/N4860.pdf)
4. cppreference.com — const_cast conversion (https://en.cppreference.com/w/cpp/language/const_cast.html)
5. cppreference.com — static_cast conversion (https://en.cppreference.com/w/cpp/language/static_cast.html)
6. cppreference.com — dynamic_cast conversion (https://en.cppreference.com/w/cpp/language/dynamic_cast.html)
7. cppreference.com — reinterpret_cast conversion (https://en.cppreference.com/w/cpp/language/reinterpret_cast.html)

8. cppreference.com — Explicit type conversion (https://en.cppreference.com/w/cpp/language/explicit_cast.html)

---

1. array-to-pointer, function-to-pointer, signed-unsigned и проч., и проч... ↩
2. примечательно, что в статье на cppreference в этом блоке содержится ошибка (там требуют, чтобы `void*` было типом приводимого выражения, а не целевым типом), которую на момент статьи нельзя исправить, потому что сайт в read-only режиме :(↩